



Factory Method e Abstract Factory

Padrões de Projeto - Aula #05

Prof. Dr. Carlos Melo

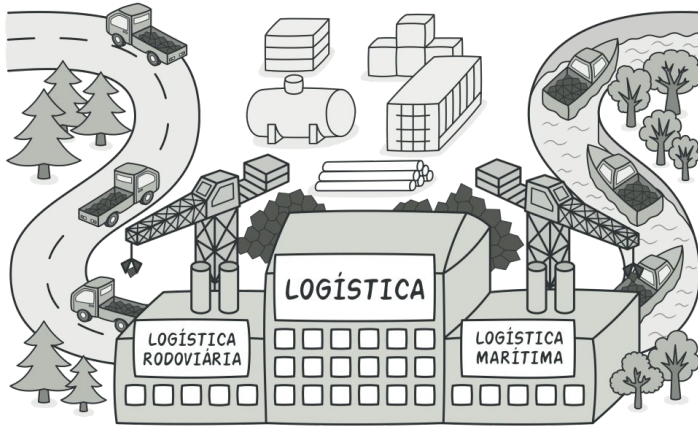
 carlos.melo@upe.br



Padrões Criacionais (parte 1)

Factory Method

Delegue à subclasse a decisão de qual classe instanciar



Abstract Factory

Crie famílias inteiras de objetos que combinam entre si

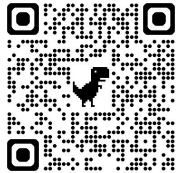


- Os padrões criacionais fornecem mecanismos para criação de objetos.
- As fábricas compõem o que é popularmente chamado de Framework e permitem decidir qual classe concreta deve ser instanciada.



Objetivos de aprendizagem

- Compreender que todo **new** ou construtor no cliente é uma forma de acoplamento.
- Distinguir Factory Method de Abstract Factory pelo número de produtos e não pelo nome.
- Implementar fábricas.
- Saber quando nenhuma das duas são necessárias
 - Em Python, uma função fábrica (HoF) pode ser uma alternativa mais simples ao Factory Method quando não há necessidade da hierarquia de *creators*.



Lá e de volta outra vez

- A solução usada para definir a forma de pagamento foi boa. Sem dúvida reduziu o acoplamento e jogou a responsabilidade de determinar a forma de pagar para o cliente

```
1 class Pedido:
2     def __init__(self, pagamento: Pagamento):
3         self.pagamento = pagamento
4
5 Pedido(Pix()).finalizar(100)
6 Pedido(Cartao()).finalizar(100)
```

- O if e else que saíram daqui foram parar lá no cliente, continua existindo.

```
1 def processar(tipo: str, valor: float):
2     if tipo == "pix":
3         pagamento = Pix()
4     elif tipo == "cartao":
5         pagamento = Cartao()
6     ...
7     Pedido(pagamento).finalizar(valor)
```

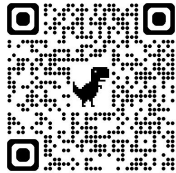


Factory Method - O Problema

- Precisamos exportar as imagens processadas pelo telescópio:

```
1 class ExportadorDeAcervo:
2     def exportar(self, imagens: list[Imagem], destino: str) -> None:
3         arquivo = ArquivoFITS(destino)
4         arquivo.escrever_cabecalho()
5         for img in imagens:
6             arquivo.adicionar(img)
7         arquivo.fechar()
8         print(f"Exportado para {destino}")
```

- FITS é o padrão usado para imagens astronômicas
- Porém, chegaram às seguintes demandas:
 - O site público precisa carregar no formato .png
 - O laboratório exige TIFF
 - O FITS deve ser comprimido antes de ser armazenado
- **ExportadorDeAcervo** só conhece **ArquivoFITS**



Soluções Ingênuas

- **if** dentro do exportar

```
1 def exportar(self, imagens, destino, formato):  
2     if formato == "fits":  
3         arquivo = ArquivoFITS(destino)  
4     elif formato == "png":  
5         arquivo = ArquivoPNG(destino)  
6     elif formato == "tiff":  
7         arquivo = ArquivoTIFF(destino)
```

- O resto segue igual.
- Outra opção seria criar um exportador para cada tipo de arquivo.

```
1 class ExportadorFITS(ExportadorDeAcervo):  
2     def exportar(self, imagens, destino):  
3         arquivo = ArquivoFITS(destino)  
4         arquivo.escrever_cabecalho()  
5         for img in imagens: ...
```

- Mas grande parte do código ficaria duplicada em múltiplos arquivos.
- E eu empurraria o **if** para outro lugar.



Factory Method - A ideia

- Buscar uma forma de resolver esse tipo de problema de uma única linha (o que instanciar) delegando para um **método** a responsabilidade de decidir quem será instanciado

```
1 class ExportadorDeAcervo:
2     def exportar(self, imagens: list[Imagem], destino: str) -> None:
3         arquivo = ArquivoFITS(destino)
4         arquivo.escrever_cabecalho()
5         for img in imagens:
6             arquivo.adicionar(img)
7         arquivo.fechar()
8         print(f"Exportado para {destino}")
```



Factory Method - Intenção

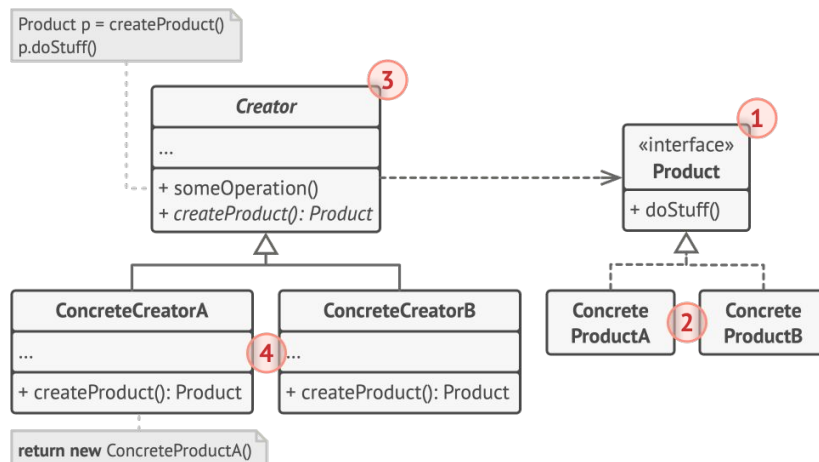
"Definir uma interface para criar um objeto, mas deixar as subclasses decidirem qual classe instanciar. O Factory Method permite a uma classe **postergar** a instanciação para **subclasses**."

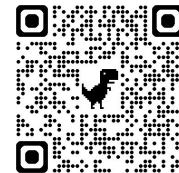
- Padrão criacional
 - Também chamado de Virtual Constructor
- Posterga a instanciação
 - A classe base não sabe, e não precisa saber;
- Subclasses
 - A decisão é feita por herança, em tempo de compilação da hierarquia.



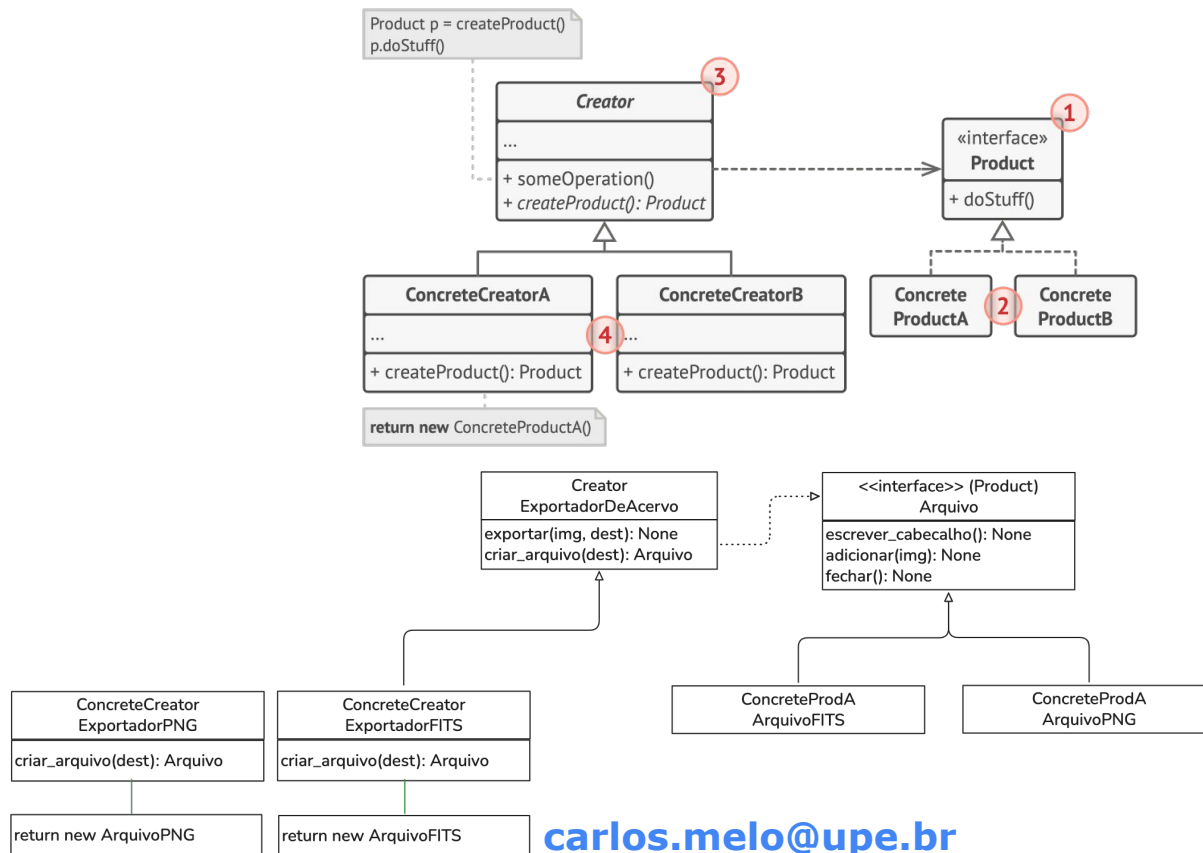
Factory Method - Estrutura

1. **Product** declara a interface que é comum a todos os objetos que podem ser produzidos pelo **Creator** ou subclasses
2. **ConcreteProductX** são implementações diferentes da interface do produto
3. **Creator** declara o Fabric Method que retorna novos objetos do produto
4. **ConcreteCreators** sobrescrevem o Fabric Method base para retornar um tipo diferente de produto





Aplicando ao nosso domínio





Aplicando ao nosso domínio

- O contrato do produto que o Exportador Produz
- **fechar()** é concreto já que nem tudo numa ABC precisa ser abstrato

```
1 from abc import ABC, abstractmethod
2
3 class Arquivo(ABC):
4     def __init__(self, destino: str):
5         self.destino = destino
6         self._itens = 0
7
8     @abstractmethod
9     def escrever_cabecalho(self) -> None: ...
10
11     @abstractmethod
12     def adicionar(self, imagem: str) -> None: ...
13
14     def fechar(self) -> None:
15         print(f"    ↳ {self.destino} fechado ({self._itens} imagens)")
16
17
18 class ArquivoFITS(Arquivo):
19     def escrever_cabecalho(self):
20         print("    ↳ [FITS] SIMPLE=T BITPIX=-32 NAXIS=2")
21     def adicionar(self, imagem):
22         self._itens += 1
23         print(f"    ↳ [FITS] HDU {self._itens}: {imagem}")
24
25
26 class ArquivoPNG(Arquivo):
27     def escrever_cabecalho(self):
28         print("    ↳ [PNG] \\x89PNG IHDR sRGB")
29     def adicionar(self, imagem):
30         self._itens += 1
31         print(f"    ↳ [PNG] convertendo {imagem} para 8-bit")
32
33
34 class ArquivoTIFF(Arquivo):
35     def escrever_cabecalho(self):
36         print("    ↳ [TIFF] II*\0 LZW CMYK")
37     def adicionar(self, imagem):
38         self._itens += 1
39         print(f"    ↳ [TIFF] IFD {self._itens}: {imagem} @300dpi")
```

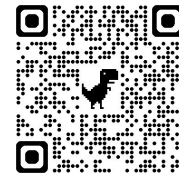


Aplicando ao nosso domínio

- Agora os três criadores são simétricos
- **ExportadorDeAcervo** é abstrato e ninguém exporta sem escolher formato

```
1 class ExportadorDeAcervo(ABC):
2     def exportar(self, imagens: list[str], destino: str) -> None:
3         arquivo = self.criar_arquivo(destino)
4         arquivo.escrever_cabecalho()
5         for img in imagens:
6             arquivo.adicionar(img)
7         arquivo.fechar()
8
9     @abstractmethod
10    def criar_arquivo(self, destino: str) -> Arquivo: ...
11
12
13 class ExportadorFITS(ExportadorDeAcervo):
14     def criar_arquivo(self, destino): return ArquivoFITS(destino)
15
16 class ExportadorPNG(ExportadorDeAcervo):
17     def criar_arquivo(self, destino): return ArquivoPNG(destino)
18
19 class ExportadorTIFF(ExportadorDeAcervo):
20     def criar_arquivo(self, destino): return ArquivoTIFF(destino)
```

Factory Method - Testando



```
1 EXPORTADORES: dict[str, type[ExportadorDeAcervo]] = {
2     "fits": ExportadorFITS,
3     "png": ExportadorPNG,
4     "tiff": ExportadorTIFF,
5 }
6
7 def exportar_acervo(imagens: list[str], destino: str, formato: str) -> None:
8     exportador = EXPORTADORES[formato]()
9     exportador.exportar(imagens, destino)
10
11 imagens = ["hubble_001", "hubble_002", "andromeda"]
12 exportar_acervo(imagens, "acervo.fits", "fits")
13 exportar_acervo(imagens, "site/galeria.png", "png")
```

```
1 Exportando 3 imagens ->
2   ├── [FITS] SIMPLE=T BITPIX=-32 NAXIS=2
3   ├── [FITS] HDU 1: hubble_001
4   ├── [FITS] HDU 2: hubble_002
5   └── [FITS] HDU 3: andromeda
6   └─ acervo.fits fechado (3 imagens)
7
8 Exportando 3 imagens ->
9   ├── [PNG] \x89PNG IHDR sRGB
10  └── [PNG] convertendo hubble_001 para 8-bit
11  ...
```

HoF



- Não preciso dos ConcreteCreators quando uso um Callable em Python

```
1 from typing import Callable
2
3 FabricaDeArquivo = Callable[[str], Arquivo]
4
5 class ExportadorDeAcervo:
6     def __init__(self, criar_arquivo: FabricaDeArquivo):
7         self._criar_arquivo = criar_arquivo
8
9     def exportar(self, imagens: list[str], destino: str) -> None:
10        print(f"Exportando {len(imagens)} imagens ->")
11        arquivo = self._criar_arquivo(destino)
12        arquivo.escrever_cabecalho()
13        for img in imagens:
14            arquivo.adicionar(img)
15        arquivo.fechar()
16
17
18 ExportadorDeAcervo(ArquivoFITS).exportar(imagens, "acervo.fits")
19 ExportadorDeAcervo(ArquivoPNG).exportar(imagens, "galeria.png")
20 ExportadorDeAcervo(ArquivoTIFF).exportar(imagens, "impressao.tiff")
```

```
1 from abc import ABC, abstractmethod
2
3 class Arquivo(ABC):
4     def __init__(self, destino: str):
5         self.destino = destino
6         self._itens = 0
7
8     @abstractmethod
9     def escrever_cabecalho(self) -> None: ...
10
11     @abstractmethod
12     def adicionar(self, imagem: str) -> None: ...
13
14     def fechar(self) -> None:
15         print(f"  ↳ {self.destino} fechado ({self._itens} imagens)")
16
17
18 class ArquivoFITS(Arquivo):
19     def escrever_cabecalho(self):
20         print("  ↳ [FITS] SIMPLE=T BITPIX=-32 NAXIS=2")
21     def adicionar(self, imagem):
22         self._itens += 1
23         print(f"  ↳ [FITS] HDU {self._itens}: {imagem}")
24
25
26 class ArquivoPNG(Arquivo):
27     def escrever_cabecalho(self):
28         print("  ↳ [PNG] \x89PNG IHDR sRGB")
29     def adicionar(self, imagem):
30         self._itens += 1
31         print(f"  ↳ [PNG] convertendo {imagem} para 8-bit")
32
33
34 class ArquivoTIFF(Arquivo):
35     def escrever_cabecalho(self):
36         print("  ↳ [TIFF] II*\0 LZW CMYK")
37     def adicionar(self, imagem):
38         self._itens += 1
39         print(f"  ↳ [TIFF] IFD {self._itens}: {imagem} @300dpi")
```



Prós e Contras do FactoryMethod

Prós

- Elimina o acoplamento entre o Creator e os produtos concretos.
- Respeita o **S** do **SOLID** já que o código de criação fica num único lugar, fácil de manter
- Respeita o **O** do **SOLID**, novos produtos entram sem tocar no código existente

Contras

- O código fica mais complicado com uma subclasse de Creator para cada produto.
- Só faz sentido se já existe (ou se justifica) uma hierarquia de creators

Não use:

- Se existe um único tipo de produto e nenhuma previsão de outros
- Se o Creator não tem operação de negócio própria
- Dependendo da linguagem usada no projeto uma HoF resolve



Abstract Factory - O Problema

- Sua equipe está escrevendo uma biblioteca de interface que roda em Android e iOS. Cada plataforma tem seu próprio visual:

	Android (Material)	iOS (Cupertino)
Botão	retangular, sombra, MAIÚSCULAS	arredondado, plano, azul
Checkbox	quadrado com ✓	switch deslizando

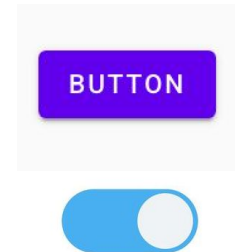
- Com o que aprendemos até agora, cada *widget* seria criado separadamente



Abstract Factory - O Problema

- Não quebra, não dá erro, até porque o problema não é o componente em si, mas sim sua família (não a sua, mas a do Android/iPhone)
- A tela simplesmente fica torta e o usuário percebe antes de você

```
1 botao = BotaoMaterial()  
2 checkbox = CheckboxCupertino()
```





Abstract Factory - Intenção

"Fornecer uma interface para criar **famílias** de objetos relacionados ou dependentes sem especificar suas classes concretas."

- Padrão criacional
- Também chamado de Kit
- Famílias correspondem a um conjunto de objetos relacionados que precisam ser compatíveis entre si.
- Nesse contexto a ideia é que o cliente nunca vai precisar escrever BotaoMaterial ou CheckboxCupertino.



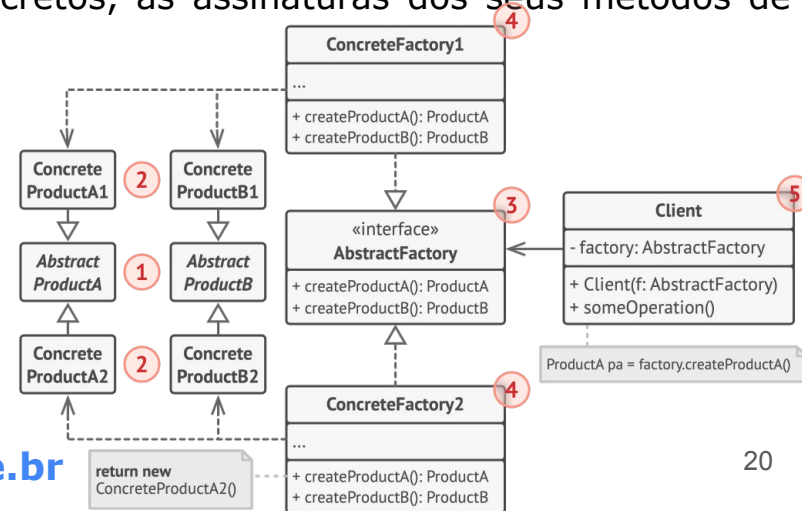
Abstract Factory x Factory Method

	Factory Method	Abstract Factory
Cria	1 Produto	Família de Produtos
Mecanismo	Herança	Composição
Métodos de criação	1	N (1 por produto)
Garante compatibilidade	Não (nem precisa, já que é um único produto)	Sim

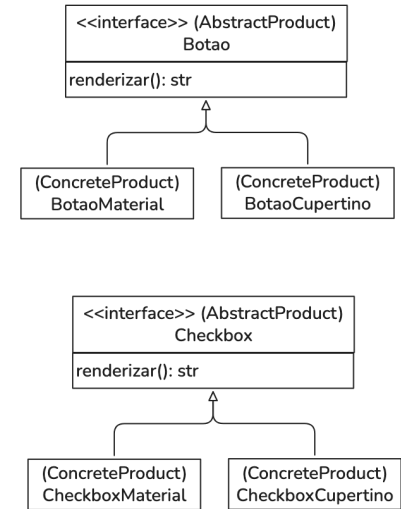
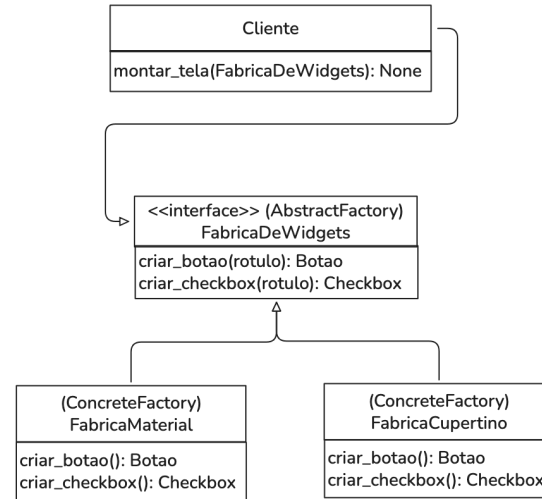
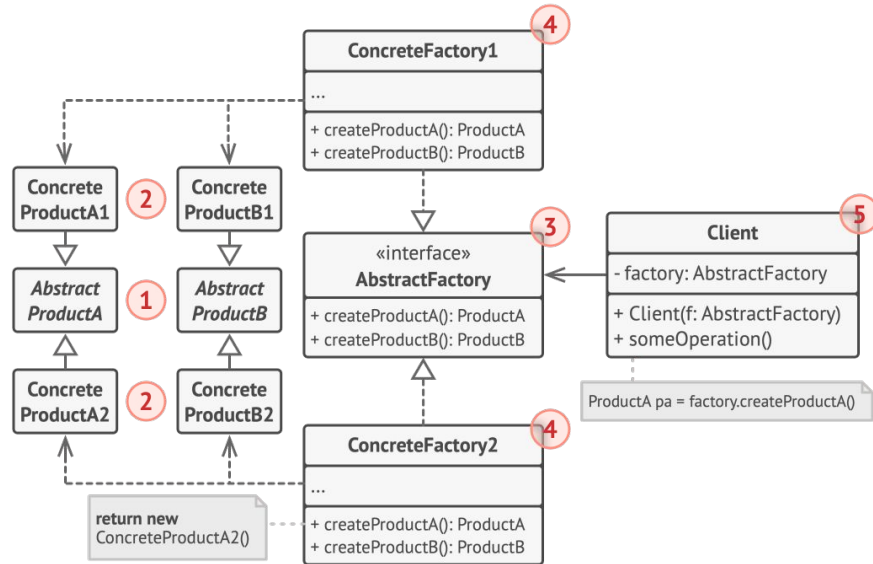


Abstract Factory - Estrutura

1. **AbstractProducts** declaram interfaces para um conjunto de Products distintos mas relacionados que fazem parte da mesma família
2. **ConcreteProducts** são várias implementações de **AbstractProducts** agrupados por variantes.
3. A interface **AbstractFactory** declara um conjunto de métodos para criação de cada um dos produtos abstratos
4. **ConcreteFactory** implementam métodos de criação de produtos. Cada Factory concreta corresponde a uma variante específica de produtos
5. Embora as fábricas concretas instanciam produtos concretos, as assinaturas dos seus métodos de criação devem retornar produtos abstratos.



Aplicando ao nosso domínio



Aplicando ao nosso domínio



```
1 from abc import ABC, abstractmethod
2
3 class Botao(ABC):
4     @abstractmethod
5     def renderizar(self) -> str: ...
6
7 class Checkbox(ABC):
8     @abstractmethod
9     def renderizar(self) -> str: ...
```

```
1 class BotaoMaterial(Botao):
2     def __init__(self, rotulo): self.rotulo = rotulo
3     def renderizar(self):       return f"[ {self.rotulo.upper()} ]"
4
5 class BotaoCupertino(Botao):
6     def __init__(self, rotulo): self.rotulo = rotulo
7     def renderizar(self):       return f"( {self.rotulo} )"
8
9 class CheckboxMaterial(Checkbox):
10    def __init__(self, rotulo): self.rotulo = rotulo
11    def renderizar(self):       return f"[x] {self.rotulo}"
12
13 class CheckboxCupertino(Checkbox):
14    def __init__(self, rotulo): self.rotulo = rotulo
15    def renderizar(self):       return f"(●—) {self.rotulo}"
```

Aplicando ao nosso domínio



```
1 class FabricaDeWidgets(ABC):
2     @abstractmethod
3     def criar_botao(self, rotulo: str) -> Botao: ...
4     @abstractmethod
5     def criar_checkbox(self, rotulo: str) -> Checkbox: ...
6
7
8 class FabricaMaterial(FabricaDeWidgets):
9     def criar_botao(self, rotulo): return BotaoMaterial(rotulo)
10    def criar_checkbox(self, rotulo): return CheckboxMaterial(rotulo)
11
12 class FabricaCupertino(FabricaDeWidgets):
13     def criar_botao(self, rotulo): return BotaoCupertino(rotulo)
14     def criar_checkbox(self, rotulo): return CheckboxCupertino(rotulo)
```

```
1 def montar_tela(fabrica: FabricaDeWidgets) -> None:
2     botao = fabrica.criar_botao("confirmar")
3     checkbox = fabrica.criar_checkbox("Lembrar de mim")
4     print(botao.renderizar())
5     print(checkbox.renderizar())
6
7
8 FABRICAS = {"android": FabricaMaterial, "ios": FabricaCupertino}
9
10 montar_tela(FABRICAS["android"]())
11 print("---")
12 montar_tela(FABRICAS["ios"]())
```

- montar_tela não menciona Material nem Cupertino.
- Trocar de plataforma é uma linha.

```
1 [ CONFIRMAR ]
2 [x] Lembrar de mim
3 ---
4 ( confirmar )
5 (●—) Lembrar de mim
```



Prós e Contras do Abstract Factory

Prós

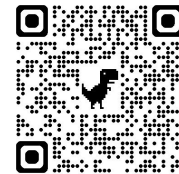
- Garante compatibilidade entre os produtos de uma família.
- Estados inválidos deixam de ser representáveis.
- Elimina o acoplamento entre cliente e classes concretas.
- Respeita o **S** do **SOLID** (Toda a criação se dá em um único lugar)
- Respeita o **O** do **SOLID** (Uma família nova é uma classe nova, o resto não muda)

Contras

- Adicionar um produto novo é caro: um método novo na interface abstrata obriga a alterar todas as fábricas concretas. Quatro perfis? Quatro edições.
- Vale a pena só quando as famílias são estáveis e os produtos realmente dependem uns dos outros.

Não use:

- Se tiver muitas interfaces e classes:
 - 3 famílias × 3 produtos = 9 classes + 4 ABCs + 3 fábricas.



Prof. Dr. Carlos Melo

<https://calendly.com/prof-casm>

 carlos.melo@upe.br